

Chapter 2: Algorithmic Paradigms: Dynamic Programming

2.1 The Dynamic Programming Paradigm

Optimal Substructure and Overlapping Subproblems

Dynamic Programming (DP) is a powerful algorithmic optimization technique that solves complex problems by breaking them down into simpler, overlapping subproblems. Unlike Divide and Conquer (where subproblems are independent, e.g. Merge Sort), DP is applied when subproblems share sub-subproblems.

Two essential prerequisites define a problem suitable for DP: 1. Optimal Substructure: An optimal solution to the overall problem can be constructed from optimal solutions of its constituent subproblems. 2. Overlapping Subproblems: The recursive formulation revisits the same subproblems repeatedly rather than generating new subproblems.

Key Formula / Law: Fibonacci DP Recurrence: $F(n) = F(n-1) + F(n-2)$ with $F(0)=0, F(1)=1$ | Naïve Recursion: $O(2^n)$
■ DP: $O(n)$ Time, $O(1)$ Space

2.2 Memoization (Top-Down) vs. Tabulation (Bottom-Up)

Design Patterns and Trade-offs

1. Top-Down (Memoization): Preserves the natural recursive definition and uses a hash table or array cache to store results of expensive function calls. If a subproblem was previously solved, return cached value immediately. 2. Bottom-Up (Tabulation): Eliminates recursion completely. Iteratively fills a DP table starting from the smallest base cases up to the target state. Tabulation typically incurs lower function-call overhead and allows memory optimization.

2.3 The 0/1 Knapsack Problem

State Formulation and Decision Trees

Given weights $w[i]$ and values $v[i]$ of N items, determine the maximum value that fits within capacity W . At each item i , we choose to either include it (if $w[i] \leq$ remaining capacity) or exclude it: $DP[i][w] = \max(DP[i-1][w], DP[i-1][w - w[i]] + v[i])$.

Key Takeaways:

- Identify state parameters that uniquely define a subproblem.
- Formulate a clean mathematical recurrence relation before writing code.
- Recognize overlapping subproblems to transition exponential brute-force into polynomial DP.
- Look for rolling array optimizations to reduce space complexity.